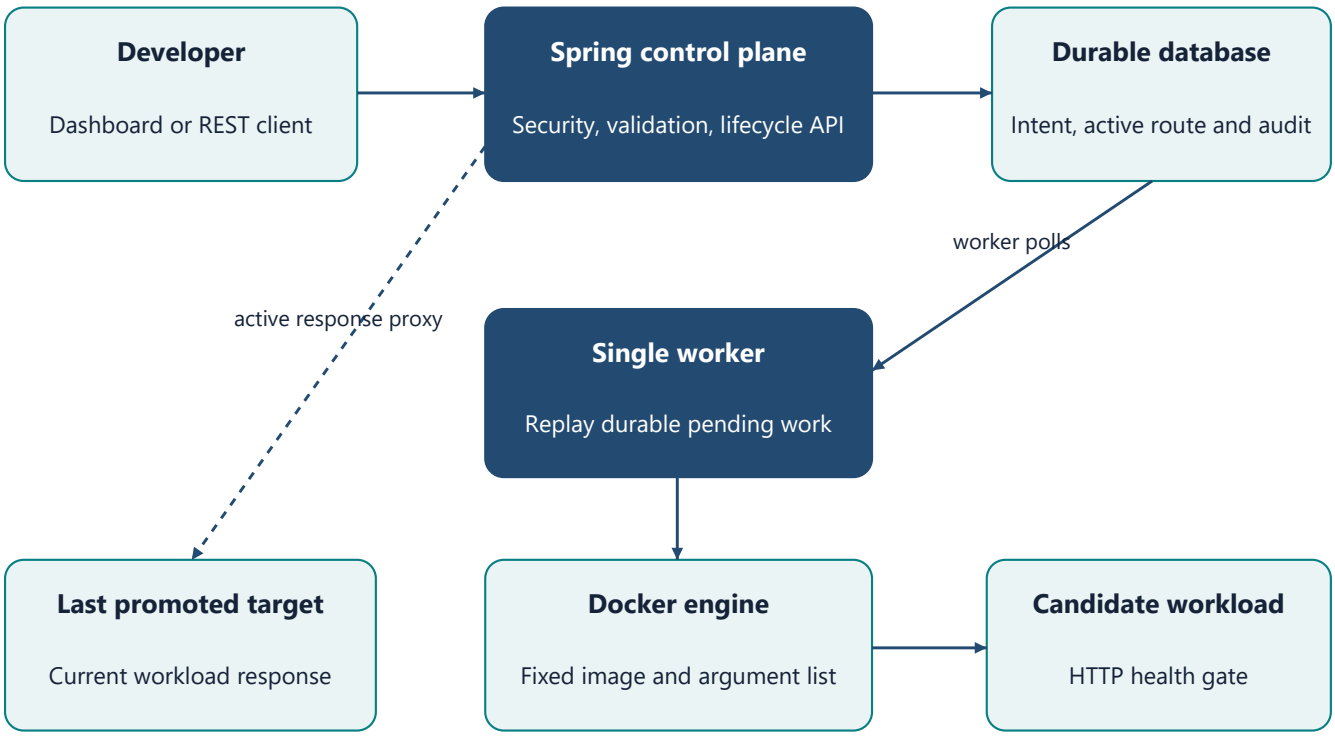
# Launchpad project handbook

## A practical guide to the deployment control plane

Prepared for Vishal Kumar. Learn the system from the developer request to the Docker workload, understand why the failure paths work, and practise explaining the design under interview follow-up questions.




SQL transaction boundaries cover durable state; Docker is reconciled separately.

A durable request becomes a real workload only after validation, runtime execution and a health gate. The database records which target the API should use.

| What has been demonstrated | Where |
|---|---|
| 17 integration tests and nine real acceptance checks | Local verification on 14 October 2026 |
| Abrupt restart recovery and Terraform lifecycle | Local failure and infrastructure scenarios |
| Tests, both image builds and live Docker acceptance | GitHub Actions run 38583306582 |

Source reviewed on 20 October 2026 at commit 7b11f3e. Historical verification is dated explicitly; this handbook does not claim a new test run today.

# How to use this handbook

First, learn the request flow and the three durable records. Next, work through failure and security boundaries. Finally, practise the question bank aloud. Use the PDF bookmarks or the links below to jump between topics.


| Topic | Pages |
|---|---|
| Project purpose and boundaries | 3 |
| Architecture and code responsibilities | 4 to 5 |
| Request sequence and deployment states | 6 to 7 |
| Database and API contract | 8 to 9 |
| Idempotency and concurrent requests | 10 to 11 |
| Health gate and rollback | 12 to 13 |
| Recovery and cleanup | 14 to 15 |
| Security and Docker networking | 16 to 17 |
| Observability and test evidence | 18 to 19 |
| Terraform and production evolution | 20 to 21 |
| Limitations and local operations | 22 to 23 |
| Screenshots and demonstration evidence | 24 to 25 |
| Question bank on product and APIs | 26 to 27 |
| Question bank on failures and security | 28 to 29 |
| Question bank on scale and cloud | 30 |
| Spoken explanations and demo script | 31 |
| Glossary and source references | 32 |


## A useful answer pattern

State the invariant, name the mechanism, walk through one failure, then state the boundary. Example: the old route must remain selected until the candidate is healthy; a SQL transaction commits the new route only after three consecutive probes; a bad candidate leaves the old target unchanged; request draining is still future work.

# The problem and the project scope

## The developer problem

A deployment is more than starting a container. A developer needs to request a release without knowing every runtime command. The platform must authorize the request, avoid duplicate work after retries, keep a useful history, and prevent an unhealthy candidate from taking over the active service route. Launchpad implements that narrow workflow end to end on a local machine.

## What a control plane means here

The control plane decides and records what should run. Its API accepts deployment intent; its worker performs the slow runtime work. The workload container is the application being run. Launchpad also includes a small response proxy for the demonstration, so the control plane participates in that sample response path. It is not a separate production ingress layer.

## Three terms to keep distinct

| Term | Meaning in this repository |
| --- | --- |
| Service | A named resource owned by one principal, with one active target. |
| Deployment | A durable request to run an allowlisted release under dev or prod approval policy. |
| Release | v1, v2 or bad-health, supplied as APP_VERSION to the same sample image. |

## The release fixture is deliberately small

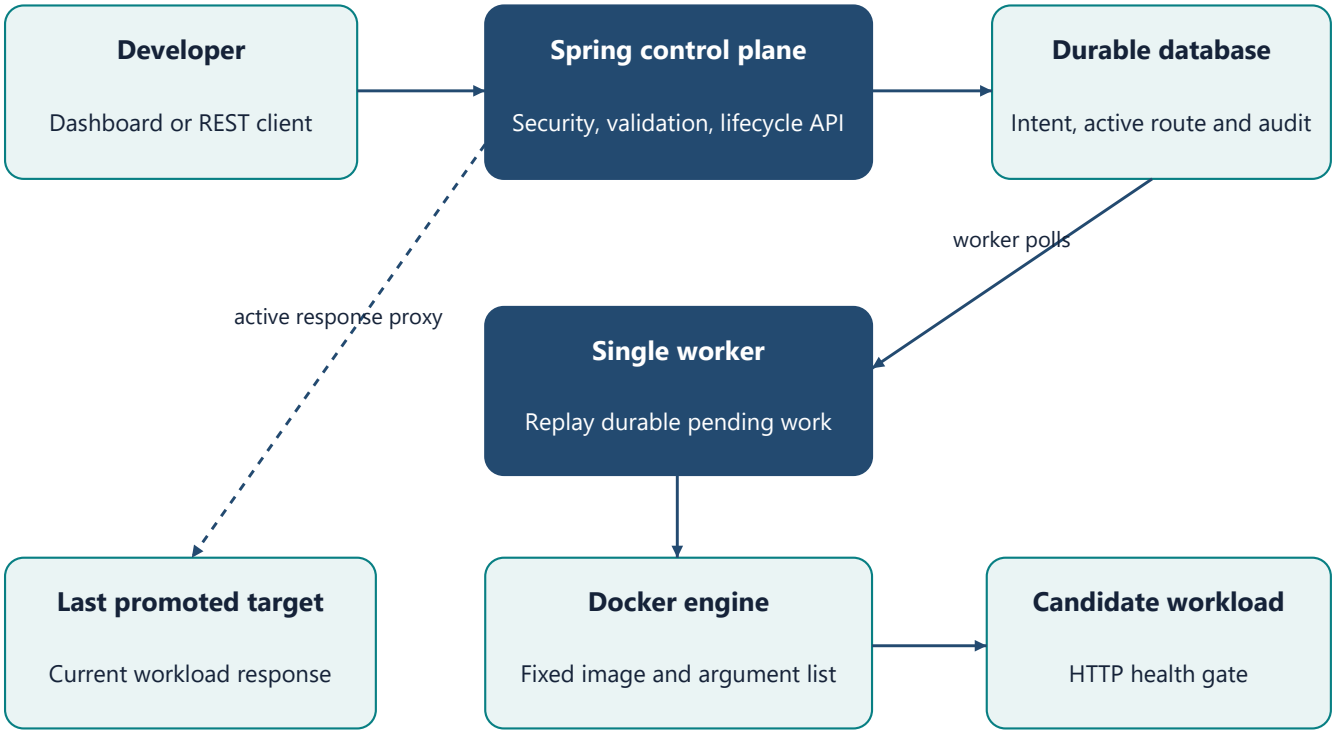
The sample Python server returns a greeting for v1 and v2, while bad-health returns 503 on /health. These values exercise the lifecycle; they are not three independently built application images. The administrator-configured image tag launchpad-workload:local can be changed locally. An allowlist and --pull=never do not make a mutable tag an immutable artifact.

## What makes the project platform engineering

It offers a repeatable developer path with guardrails, resource ownership, approval policy, runtime isolation and operational evidence. The differentiating work is the lifecycle and its failure boundaries. There is no Kubernetes cluster, deployed Azure control-plane adapter, LLM workflow or production availability measurement in the current project.

Explain it as a verified local deployment platform with a clear path to production hardening, rather than as a finished enterprise platform.

# The system at a glance




SQL transaction boundaries cover durable state; Docker is reconciled separately.

The API persists intent before the worker performs Docker operations. A dashed line represents the demonstration response proxy. Runtime and SQL effects are not one transaction.

## The synchronous boundary

The request thread authenticates, validates, locks and stores intent. It returns HTTP 202 with a deployment ID. The client can poll status and audit events instead of holding a connection through container startup and health checks.

## The asynchronous boundary

A scheduled worker reads QUEUED and DEPLOYING rows. It claims or resumes an intent, ensures a deterministic candidate exists, probes it, then commits a result. Slow Docker and HTTP calls occur outside SQL transactions.

## The durable boundary

File-backed H2 stores services, deployments and audit events; Flyway creates the schema. The current design assumes one process and one worker. The database is also the work queue, so no separate broker is required for this lab. Its role is durable coordination, not just CRUD storage.

# Where each responsibility lives

| Component | Responsibility | Start reading here |
|---|---|---|
| LaunchpadApplication | Bootstraps Spring and scheduling | main and @EnableScheduling |
| SecurityConfig | Bearer identity and route role checks | security and the token filter |
| ApiController | JSON validation and HTTP endpoints | Register and Deploy records |
| ControlPlane | Ownership, quota, idempotency and policy | submit, approve and cancel |
| Store | SQL queries and record mapping | lockedService, byKey and status |
| DeploymentWorker | Claim, health gate, promotion and cleanup | execute and cleanupRetired |
| WorkloadRuntime | Runtime abstraction | ensureStarted, healthy, remove, response |
| DockerRuntime | Docker CLI and bounded HTTP requests | ensureStarted and command |
| ApiErrors | Application error translation | ProblemDetail response |
| Static dashboard | Token input, forms, polling and timeline | app.js refresh and api helpers |

## Why JDBC and TransactionTemplate

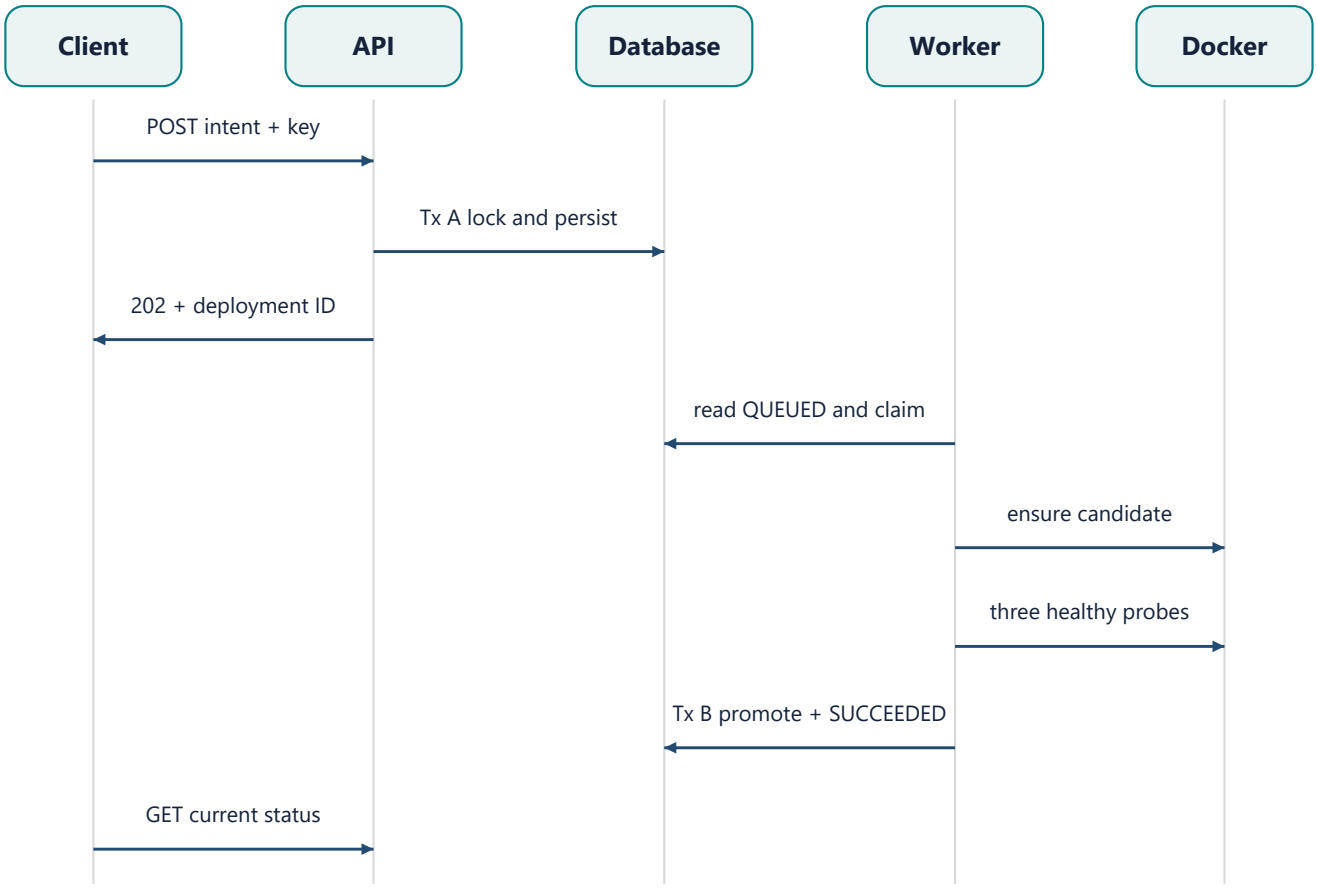
The important behavior is explicit SQL locking and a small set of state changes. JdbcTemplate keeps those operations visible. TransactionTemplate groups them into a commit or rollback without keeping transactions open around Docker calls. Store is a Spring repository proxy; internal callers use Store.jdbc() rather than reading a field from the proxy.

## Dependency inversion in practical terms

DeploymentWorker depends on WorkloadRuntime. Production wiring uses DockerRuntime; integration tests inject a deterministic fake implementation. A future Azure adapter would need more than a class name: its operation IDs, health semantics, timeout handling and compensation must be designed and verified.

Read this order when explaining code: API validation, ControlPlane.submit, Store.lockedService, DeploymentWorker.execute, then DockerRuntime.ensureStarted.

# One deployment from request to response




The response acknowledges persisted intent. The worker can start soon after commit, so the returned object and a subsequent GET can show different valid states.

## Walk the request through the implementation

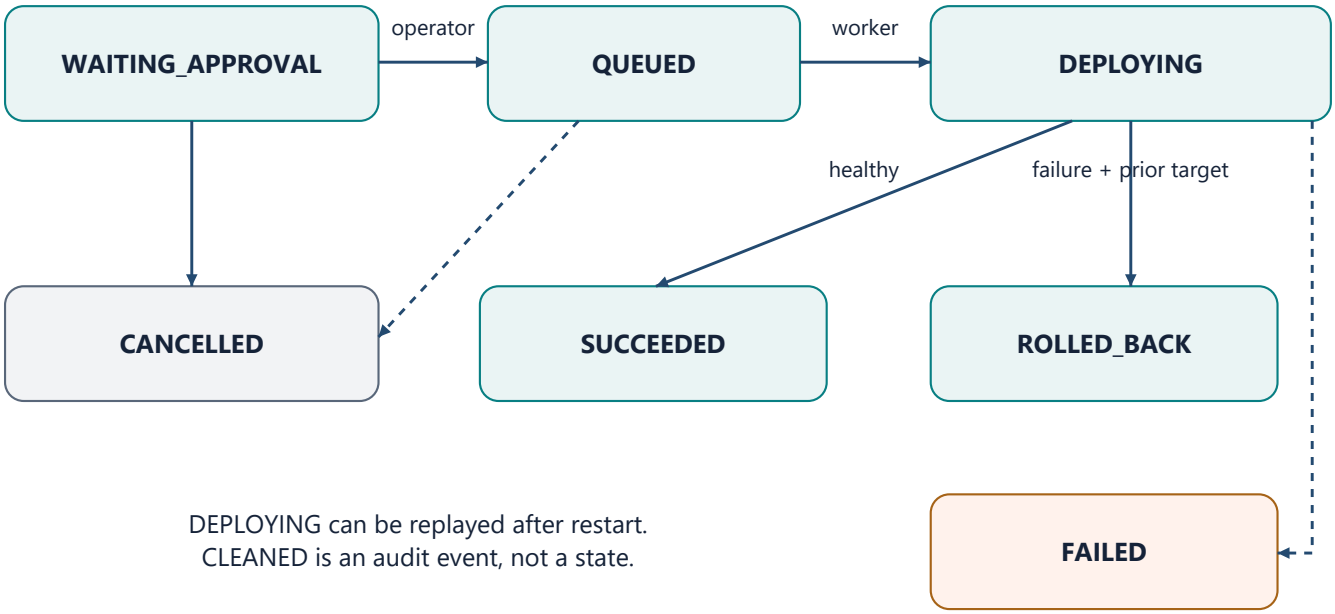
The developer posts release v1 and environment dev with an Idempotency-Key. ApiController validates the JSON and header. ControlPlane authorizes the service, locks its row, checks prior key usage and rejects another in-flight intent. It creates the deployment UUID, candidate name and REQUESTED audit event in one transaction.

The worker later records DEPLOYING, starts or reuses the named candidate, and requires a healthy streak. In a second transaction it updates active_release, active_container and active_port together with SUCCEEDED and its event. It then retires old or failed containers through cleanup reconciliation.

## What the client actually sees

The dashboard polls every three seconds. This is UI refresh, not the worker schedule. The worker uses a one-second fixed delay after a tick finishes. A long deployment can therefore delay later intents even when their API requests have already returned 202.

# The deployment state machine

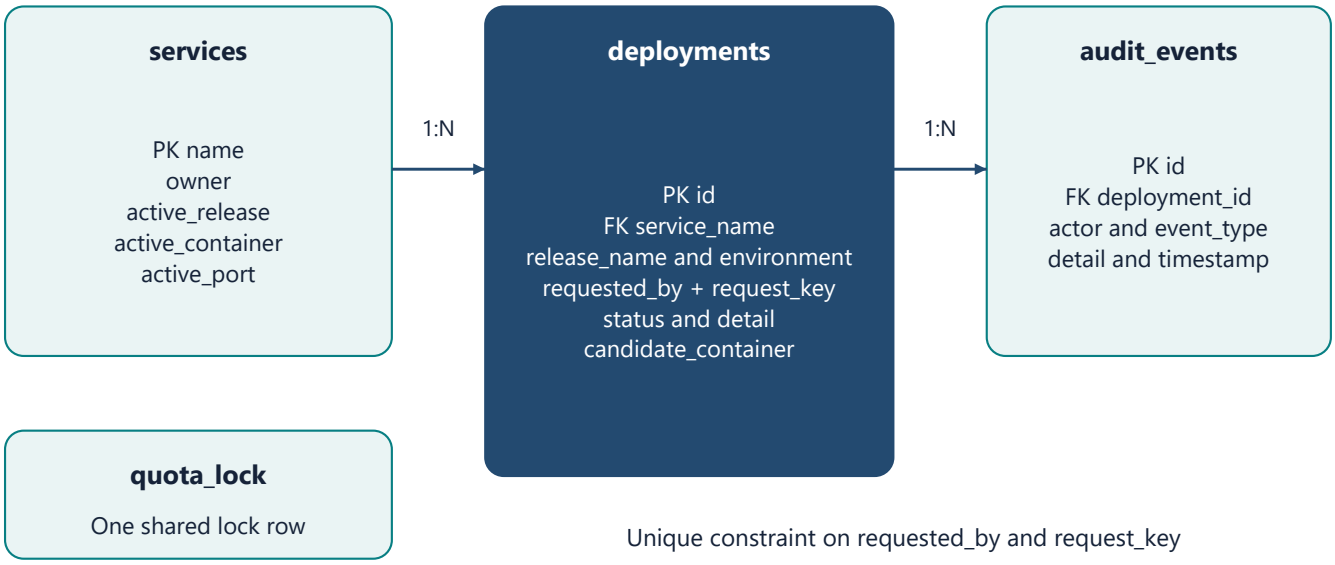



Every transition is explicit. A production approval changes WAITING_APPROVAL to QUEUED; it does not skip the health gate.

| State | Meaning | Can an owner cancel |
|---|---|---|
| WAITING_APPROVAL | Persisted prod intent awaiting an operator | Yes |
| QUEUED | Ready for a worker to claim | Yes |
| DEPLOYING | Candidate execution or restart reconciliation | No |
| SUCCEEDED | Health gate passed and target promoted | No |
| ROLLED_BACK | Failure while a prior active target exists | No |
| FAILED | Failure without a prior active target | No |
| CANCELLED | Pending work cancelled before execution | Already terminal |

CLEANED, APPROVED and RECONCILED are audit event types, not deployment states. State does not encode every external fact: ROLLED_BACK means a prior target remained selected; it does not prove that target is still healthy at that instant. Missing containers after manual cleanup can leave a historical active target until another deployment changes it.

# The database is the coordination model




One service has many deployment intents; one intent has many audit events. The key uniqueness rule is scoped to the authenticated requester, across all of that requester's services.

| Record | Why it exists |
|---|---|
| services | Authoritative ownership and active target. name is globally unique; there is no separate tenant identifier. |
| deployments | Durable work queue and history, including request parameters, state and candidate identity. |
| audit_events | Ordered per-deployment timeline with actor, event type, detail and timestamp. |
| quota_lock | One fixed row serializes service registration while the five-service quota is checked. |

## Constraints and indexes that matter

A unique constraint on (requested_by, request_key) protects retry identity even when two transactions lock different service rows. Foreign keys preserve service and deployment relationships. A status and created_at index supports queue reads; a deployment_id and id index supports timeline reads. Event IDs determine timeline ordering.

## A deliberate limitation

Busy checks and cleanup still read historical records into memory. There is no history retention policy, archive, cleanup task table or dedicated durable attempt count. Those omissions are acceptable for the lab, but matter before a long-running service handles large histories.

# The API contract and error model

All paths below are under /api. Supply an explicit bearer token. Service registration and deployment submission require application/json; unknown JSON properties are rejected.

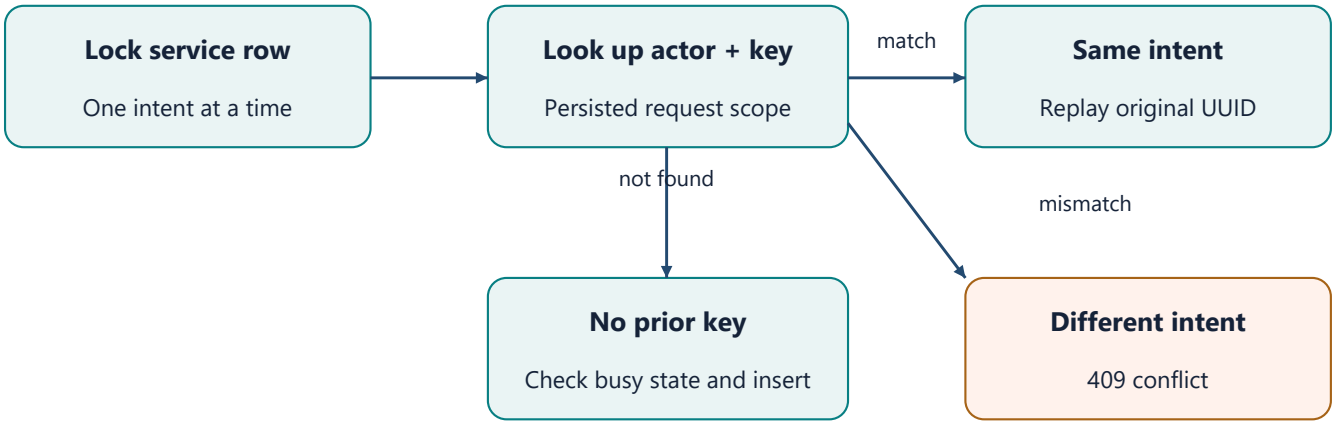
| Method | Path | Result |
|---|---|---|
| GET | /me | Identity and operator flag |
| GET / POST | /services | Owned catalog / register service with 201 |
| GET | /services/{name} | Service and active target |
| GET / POST | /services/{name}/deployments | History / persist intent with 202 |
| GET | /deployments/{id} | Current durable state |
| POST | /deployments/{id}/approve | Operator approval |
| POST | /deployments/{id}/cancel | Owner or operator cancellation |
| GET | /deployments/{id}/events | Ordered audit timeline |
| GET | /services/{name}/response | JSON from the active workload |

## Interpret failures precisely

| Status | Interpretation |
|---|---|
| 400 | Invalid body, unknown property, missing or malformed key. |
| 401 / 403 | Missing or invalid identity / authenticated identity lacks required role. |
| 404 | Resource is missing or outside a developer's ownership scope. |
| 409 / 422 | Conflicting key or busy/wrong state / five-service quota exceeded. |
| 502 / 503 | Previously selected workload unavailable / no active target exists. |

ApiException becomes a Spring ProblemDetail. Security failures return a small application/problem+json response containing status and detail. The implementation is useful standardized error handling, but it does not populate a full custom problem taxonomy or correlation ID on every failure.

# Idempotency protects request intent




The service lock handles requests for one resource. The actor and key constraint closes a different race: simultaneous reuse of one key across different resources.

## A retry after an ambiguous response

Imagine the insert commits, but the client never receives the HTTP response. Retrying with the same key, service, release and environment returns the original deployment UUID and its current state. Reusing the key for a different intent returns 409. The response may now be SUCCEEDED rather than the original QUEUED state; replay identity is stable, response state is current.

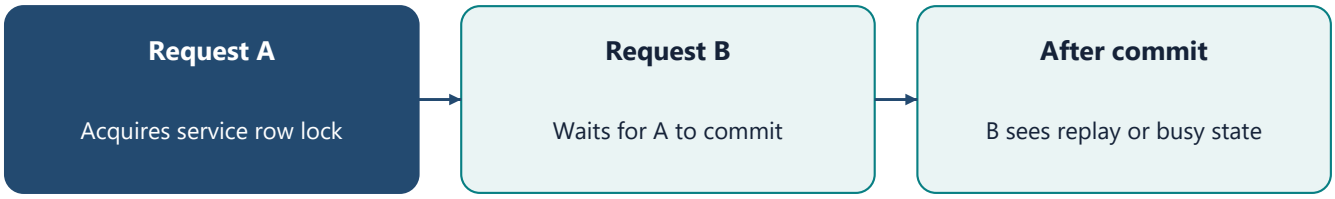
```
BEGIN
  SELECT service FOR UPDATE;
  find previous by authenticated actor and request key;
  if found: compare service, release and environment;
  otherwise: reject busy service and insert intent + event;
COMMIT
```

## What this does and does not guarantee

It deduplicates intent creation, not all side effects forever. Runtime operations can be retried after a crash, and metrics can reset or miss an increment. There is no key-expiry job; uniqueness remains as long as the deployment row remains. The dashboard retains a key for retries of the same intent while the tab remains open, but a reload loses that in-memory key.

Interview phrase: I made creation idempotent at the durable intent boundary, and made runtime execution replayable using the candidate identity.

# Concurrency is controlled at two levels




The Java synchronized worker protects one JVM; the database lock protects request races.

A row lock is held only through the decision and SQL changes. A request waiting on a lock does not get to decide using stale in-flight state.

## Same service submissions

submit locks the services row before checking key replay and busy state. If request A inserts an intent, request B waits and then observes it. B either replays the same intent or rejects a new one as busy. Approval, cancellation and the worker claim use the same resource lock, which serializes their decisions against submission.

## Service registration quota

Registration locks quota_lock row 1 before counting the owner's services and inserting. This prevents two requests from both consuming the final slot. The tradeoff is global contention: registrations by unrelated owners also wait on the same row. Per-owner lock rows or a transactional quota counter would improve that boundary.

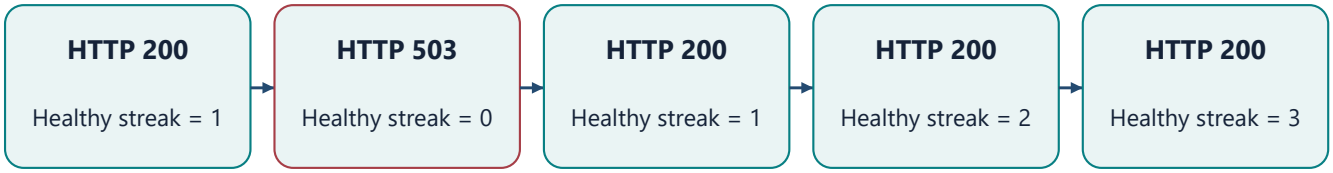
## Worker exclusion is narrower

tick and execute are synchronized methods in one Java process. The scheduled loop processes deployments serially. This is simple, but a slow job delays other services. synchronized is not a distributed lease, and embedded H2 file locking is not a design for active-active workers.

## Why not hold a transaction through Docker

A slow image runtime or probe should not hold database locks for tens of seconds. Claim state in one short transaction, perform external work without SQL locks, then commit the result in another short transaction. A crash between those steps is handled through reconciliation, not by pretending Docker joined the SQL transaction.

# The health gate before promotion




Illustrative probe sequence A failure resets the streak; the fifth response permits promotion.

This sequence is illustrative, not a recorded benchmark. Each failure restarts the streak at zero.

| Setting | Current default | Purpose |
|---|---|---|
| Required streak | 3 HTTP 200 responses | Avoid promoting on one transient success |
| Maximum attempts | 60 | Bound repeated probe work |
| Delay between probes | 500 ms | Space out attempts |
| Health deadline | 30 seconds | Bound the post-start gate |
| HTTP connect / read | 2 seconds each | Bound individual network operations |
| Docker CLI command | 60 seconds | Bound each external process call |

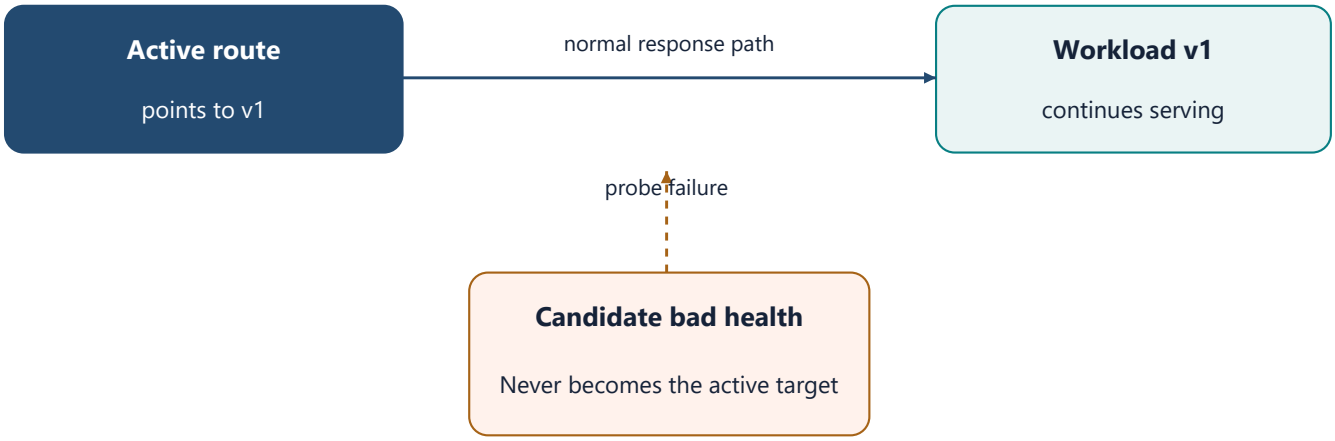
## Read the timing policy carefully

The deadline begins after ensureStarted returns the candidate's port. A probe that starts before the deadline may finish after it, and the last sleep can also extend wall-clock time. Several Docker CLI operations precede the gate, each with its own timeout. Therefore 30 seconds is not an end-to-end deployment SLA, and a healthy cold start can legitimately fail the gate on a slow host.

## What a health endpoint proves

The sample server's /health proves its fixture is ready to return the demo response. It does not test a real database dependency, migration compatibility, customer traffic or performance under load. A production readiness contract must reflect the workload's dependencies without making every transient dependency problem trigger an unsafe rollout.

# Rollback preserves the selected target




Commit ROLLED_BACK, keep v1 target unchanged, then remove only the failed candidate.

The failed candidate never replaces the service target. Launchpad does not need to restart v1 to retain the route to it.

## A failed upgrade

While bad-health is tested, services.active_container and active_port still point to v1. On failure, the worker sees that an active container field exists, records ROLLED_BACK and leaves the target unchanged. It later removes the failed candidate. A failed first deployment becomes FAILED because there is no prior target to preserve.

## A successful upgrade

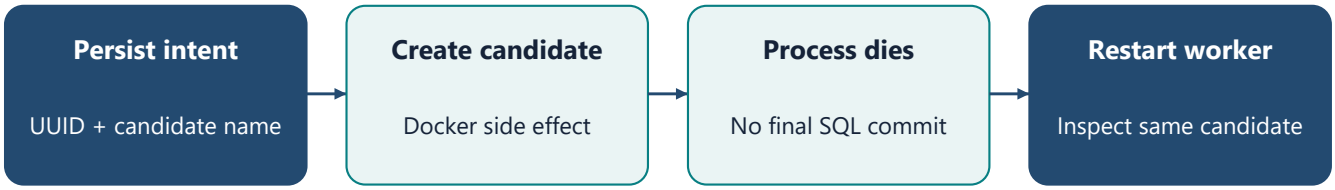
After the healthy streak, one SQL transaction updates all active target fields and records SUCCEEDED. Cleanup then retires the old container. A request that read the old port just before retirement can fail; the demo has no connection draining. This is why zero-downtime deployment is not an established project claim.

## The boundary interviewers may test

The worker does not probe the old active target before declaring ROLLED_BACK. It preserves the previously promoted route, which may itself have become unhealthy. It also does not undo database schema changes, outbound messages or arbitrary application side effects. Backward-compatible releases and migration strategy are separate production design requirements.

A precise claim is health-gated promotion with preservation of the prior target on candidate failure. Avoid claiming complete application rollback or continuous self-healing.

# Recovery across a process crash




Repeat the health gate and commit promotion. Deterministic identity makes replay converge; it does not make every operation execute once.

The restart test kills an exact child Java process while the durable intent is DEPLOYING, then opens the same database and checks the same real container ID.

| Crash window | What a restarted worker does |
|---|---|
| Before intent commit | No durable request exists; the client retries with its key. |
| After intent commit | Find QUEUED work on a later scan. |
| After claim before run | Resume DEPLOYING and create its named candidate. |
| After Docker run | Inspect namespace, start or reuse that same name, then probe. |
| After promotion commit | Keep the persisted active route and reconcile retirement. |
| During removal | Retry terminal cleanup until removal is confirmed. |

## Why deterministic names matter

ControlPlane creates namespace + deployment UUID once and persists it as candidate_container. Replaying does not generate a new name. DockerRuntime inspects the object, checks its namespace label and reuses it. The RECONCILED event exposes the resumption rather than hiding it.

## What was actually verified

The recovery script observed DEPLOYING, killed the child server, restarted with the same H2 file, asserted the same Docker container ID, checked RECONCILED and SUCCEEDED, and called the healthy workload. That is a demonstrated crash window. It is not exhaustive verification of every kill point or graceful shutdown path.

# Cleanup and external side effects

## History doubles as the cleanup ledger

cleanupRetired scans each service's deployment history. A terminal candidate is eligible if it is not the current active container and its events do not already include CLEANED. The runtime checks its namespace before removal. Only a confirmed removal creates the CLEANED event; an error is logged so the next scan can try again.

```
for each service and historical deployment:
  if terminal and candidate is not the current active target:
    if no CLEANED event:
      inspect ownership and remove candidate
      append CLEANED only after success
```

## Absence is a valid cleanup outcome

A missing container is already removed. The isolated acceptance helper also handles a race where the killed worker already dispatched removal: it confirms eventual absence instead of treating every nonzero rm exit as a leak. A daemon failure must remain distinguishable from a missing object.

## Reconciliation is not unlimited automatic retry

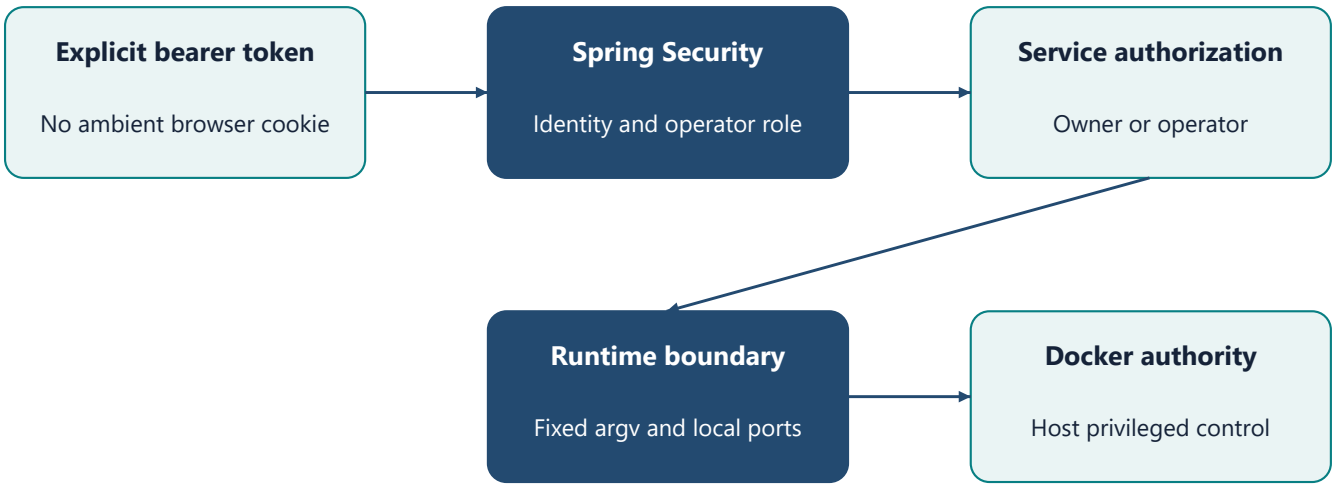
A live QUEUED or DEPLOYING row can be resumed after interruption. An ordinary Docker or health exception normally produces FAILED or ROLLED_BACK; that deployment is not automatically retried as a new deployment. The client submits a new intent with a new key after addressing the cause. Cleanup, in contrast, keeps retrying while it lacks a CLEANED event.

## Edges still worth hardening

A timed-out CLI process does not prove the daemon never created a container. Graceful interruption inside DockerRuntime is wrapped differently from interruption during the worker's sleep. Ancillary metric failures also sit inside the worker's broad success/failure try block. Production hardening should test these boundaries and separate committed promotion success from later instrumentation failures.

At scale, use indexed pending cleanup tasks and backoff rather than scanning all history and event lists every tick. Keep that change separate from the current demonstrated implementation.

# Identity roles and ownership




A restricted workload does not make Docker daemon access a restricted capability.

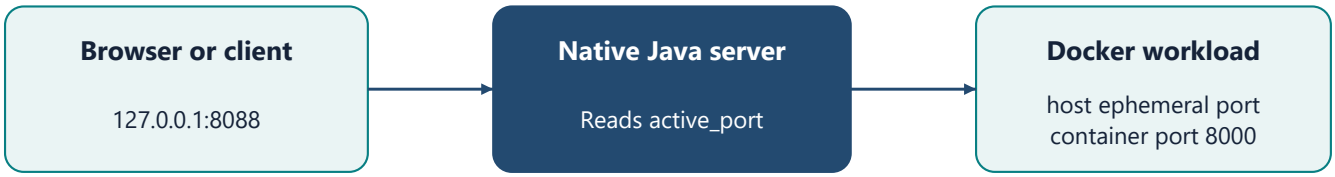
Authentication identifies the caller; route role checks and resource ownership authorize actions. The Docker engine remains a separate, powerful trust boundary.

| Action | Developer | Operator |
|---|---|---|
| List and inspect services | Own resources | All resources |
| Register a service | Owned by caller | Owned by caller |
| Request deployments | Owned service | Any service |
| Approve production intent | Denied | Allowed |
| Cancel pending work | Owned service | Any service |
| Scrape Prometheus | Denied | Allowed |

Three static tokens resolve to vishal, other-team and platform-ops. The filter compares explicitly supplied bearer bytes and stores a Spring Security Authentication for the request. Tokens must be distinct and at least 24 characters. This is a lab identity mechanism, with no expiry, rotation endpoint, JWT validation or identity-provider discovery.

Ownership is fixed at registration. Cross-owner developer reads return 404 to avoid disclosing resource existence. Operators have broad access, and an operator can approve an intent they requested; this is role separation, not an enforced two-person approval rule. A prod field on an intent does not prevent a later dev-policy request from replacing that same service slot.

# Docker isolation and host networking




With a bridged control plane container, 127.0.0.1 is that container itself. The native host path is the verified full Windows runtime.

The API response endpoint proxies to a port stored in SQL. It does not accept an arbitrary user-provided URL.

| Runtime choice | What it achieves | What it does not achieve |
|---|---|---|
| UID 10001 and read-only root | Reduce default write and privilege scope | Do not remove daemon authority |
| Drop all capabilities | Remove optional kernel capabilities | Do not eliminate kernel vulnerabilities |
| No new privileges | Prevent privilege gains through exec | Is not a complete sandbox |
| 64 MiB and 0.25 CPU | Constrain each API-managed workload | Do not cap total platform demand |
| 64 PIDs | Bound process proliferation | Do not provide tenant-level fairness |
| Loopback ephemeral ports | Avoid normal external host ingress | Local host processes can connect |
| Fixed image and argv | Prevent client-selected commands or mounts | Local image tags are still mutable |

## Why native Java is the Windows golden path

The worker publishes workload ports on the Docker host and connects to host loopback. A bridged control-plane container sees its own loopback instead. The documented Linux host-network example also needs Docker socket access and suitable group permissions. The packaged image smoke check disabled the worker; it proved health, identity, the API contract and non-root packaging, not the full containerized adapter path.

The local Terraform module uses an immutable image ID and cpu_shares = 256. CPU shares are a relative scheduling weight; they are not the same setting as the API runtime's hard 0.25 CPU limit.

# Observability and operational diagnosis

| Signal | Current implementation | How to use it |
|---|---|---|
| Control-plane health | Public /actuator/health with details hidden | Check that the service and configured health contributors respond |
| Workload probes | Candidate /health during deployment | Explain the gate decision, not continuous availability |
| Outcome counter | launchpad.deployments with result tag | Count in-process success and failed outcomes |
| Duration timer | launchpad.deployment.duration | Observe the timed worker section around runtime work |
| Queue gauge | launchpad.queue.depth | Count QUEUED plus DEPLOYING rows |
| Audit events | Persisted UUID, actor, type, detail, time | Reconstruct decisions after restart |
| Application logs | Deployment, service, outcome and bounded reason | Locate runtime errors and pending cleanup |

## Two health questions with different answers

An UP control-plane endpoint does not mean the Docker daemon or every active workload is healthy. The control plane can still accept requests and read its database while an active workload is unavailable. A response API call to a previously selected but unreachable workload returns 502. There is no continuous watchdog that automatically restores or replaces that workload.

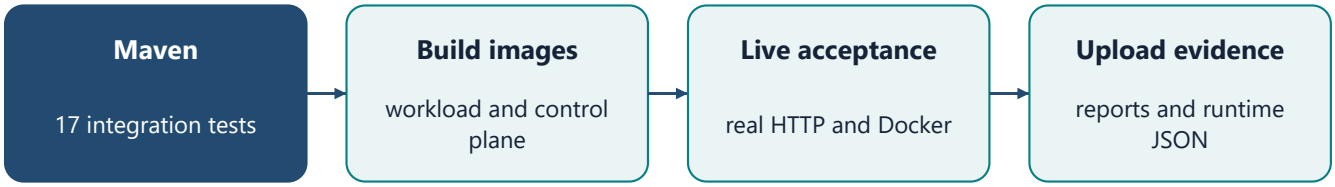
## The numbers are not durable accounting

Counters and timers live in the process and reset after restart. A crash after the SQL promotion but before a counter increment can leave a successful deployment without that metric update. The timer is not the client's queue-to-completion time; it starts after claim. Use durable rows and timestamps for historical accounting, and instrument queue wait separately before defining an SLO.

## Useful next signals

Add oldest pending age, approval wait, cleanup backlog, worker heartbeat and per-stage latency. Use bounded labels such as outcome and runtime type, rather than UUIDs or unbounded service names on every metric. Distributed tracing, alert delivery, dashboards and a measured error budget are future work, not current project features.

# What the tests actually establish




Remote GitHub Actions passed this pipeline for source commit cd73304 on 14 October 2026. The exact run is linked in the source references.

| Layer | Coverage | Boundary |
|---|---|---|
| 17 integration tests | Spring API, real in-memory H2, transactions and a fake runtime | Does not prove Docker networking |
| Nine live acceptance checks | Real HTTP, Docker containers, failures, approvals and metrics | Functional scenarios, not load testing |
| Abrupt restart scenario | Same file DB and real container ID reused after kill | One important crash window |
| Container package smoke | Non-root package, health, identity and API contract | Worker was disabled |
| Terraform local lifecycle | Create, health check, no-change plan and destroy | Independent golden workload |
| Azure reference validation | Provider and configuration schema accepted | No cloud plan or apply |

## Know the integration test groups

Authentication and ownership tests cover anonymous rejection, cross-team reads and writes, timelines and operator-only metrics. Contract tests reject malformed and unknown input. Lifecycle tests cover idempotency, concurrent replay, busy service conflicts, approval, cancellation, quota, promotion, first failure, rollback, reconciliation, Docker failure, flapping health and retryable cleanup.

## How to present the evidence

Say exactly what ran and where. The local record and GitHub result are dated 14 October 2026. This handbook rechecked the source on 20 October; writing it did not rerun the whole platform. The remote workflow does not execute the separate abrupt-restart script or Terraform lifecycle. Test counts are verified outcomes, not business impact or performance metrics.

Examples of work still to add: database failure during final commit, stale-worker races, graceful shutdown at every external call, concurrent key reuse across services, prolonged daemon outage, draining and history retention. A good test suite demonstrates defined behavior; it does not prove every possible failure safe.

# Terraform and the Azure boundary

| Aspect | Local Docker module | Azure reference |
|---|---|---|
| Provisioned resource | One independent golden workload | Resource group, B1 plan and Linux Web App |
| Artifact reference | Validated sha256 image ID | Published image digest required |
| Exposure | 127.0.0.1 port 8099 by default | Public HTTPS workload endpoint |
| Security settings | Non-root, read-only, capability restrictions | TLS minimum, HTTPS-only, system identity |
| Health behavior | HTTP checked during verification | App Service health path configured |
| Evidence | Apply, HTTP 200, no drift, destroy | Init and provider validate only |
| Control-plane integration | Not called by the worker | No Azure runtime adapter implemented |

## Why two owners must not manage one container

The API runtime and Terraform demonstrate the same workload contract independently. They use different namespace labels and names. If Terraform and the worker both owned one container, one could replace or remove it while the other believed it was active. Pick one authoritative owner for each resource and make handoffs explicit.

## What the local no-drift check means

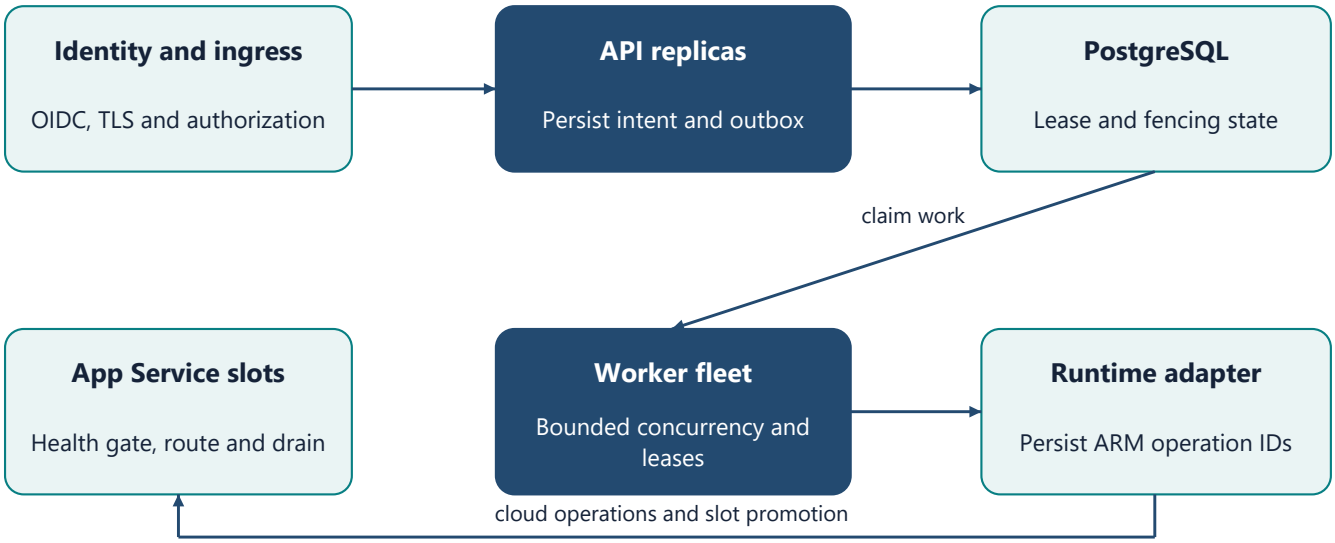
After apply, the HTTP endpoint returned a healthy greeting. A detailed plan returned exit code 0 with no changes, then destroy removed the resource. Docker defaulted memory_swap to 128 MiB, so that value was set explicitly to avoid perpetual drift. This tests convergence for this local fixture, not a cloud deployment strategy.

## What an Azure implementation would require

App Service does not provide the host Docker daemon expected by this adapter. A real Azure runtime would use managed identity and ARM APIs, store long-running operation IDs, poll operations, check the target slot, promote routing and reconcile failures. Private ACR also needs its own identity-based pull configuration. The B1 reference can incur cost; none was provisioned for this project.

# A credible path to production

**PROPOSED PRODUCTION EVOLUTION / NOT IMPLEMENTED**




The runtime API must reject stale fencing tokens or serialize effects; a SQL flag alone cannot fence an external cloud operation.

This is a design proposal, not a diagram of deployed functionality. Start with the current invariants, then add distributed coordination and a cloud-specific runtime contract.

## Separate serving from orchestration

Use a dedicated ingress and workload serving path so control-plane unavailability does not stop normal application traffic. Add connection draining around promotions. Model release artifacts by immutable digest and require compatibility checks for migrations and dependencies.

## Make work ownership explicit

Move durable state to PostgreSQL with migrations and backup policy. Claim eligible jobs transactionally, store worker identity, lease expiry, attempt count and a monotonically increasing fencing token. A heartbeat renews the lease. Every status update must prove ownership. External effects need equivalent stale-worker rejection or serialized runtime operations.

## Grow the runtime contract rather than rename it

An asynchronous cloud API needs submit, inspect, health, promote and remove semantics with persisted remote operation IDs. A broker can wake workers efficiently, but the database remains authoritative. If using a broker, an outbox avoids a gap between committing intent and publishing its notification. Test API crash, expired lease and partially completed cloud operations before scaling workers.

# The limitations worth explaining well

## One worker limits fairness and throughput

A long health gate blocks later intents for other services because the scheduled loop is serial. Fixed delay means the next scan begins after the current tick completes and another second elapses. Do not describe this as a job starting within one second. First add queue-age metrics, then bounded per-service concurrency and durable leases.

## A history scan is not a scalable queue strategy

Submission scans service history to find a busy state. Cleanup scans services, deployments and event lists. Queue order uses created_at without a tie-breaking ID. Add targeted existence queries, stable ordering, retention, indexed pending cleanup records and an explicit one-active-intent invariant before expecting high cardinality.

## Approval policy needs a protected resource model

An intent says dev or prod, but the service has one active slot. A later dev intent can replace the same service without production approval. Operators can also approve their own requests. Protected environment resources and optional requester-versus-approver constraints would be needed for stronger governance.

## The runtime trust model is local

Labels prevent accidental cross-namespace cleanup; they do not defend against a host administrator who can spoof labels or replace images. A matching namespace is not image attestation. The configured tag is mutable. Use digest pinning, artifact verification and a constrained remote runtime rather than broad host daemon access for untrusted tenants.

## Failure domains are not fully separated

H2, the worker and the response proxy share one process/host boundary. The code does not continuously revalidate active targets, drain old requests or restore lost database files. Promotion instrumentation should also be isolated from deployment failure handling. Be ready to name these limitations and explain the next test that would reduce each uncertainty.

# Run and diagnose the local project

From the launchpad project directory, start Docker Desktop in Linux-container mode. The native Java path is the demonstrated full Windows deployment route. A first build requires dependency and base-image network access.

```
# Builds workload, runs tests, packages and starts Java
./scripts/start.ps1

# Reuse the existing jar and workload image
./scripts/start.ps1 -SkipBuild

# Independent real scenarios on separate ports
python scripts/acceptance.py
python scripts/recovery.py

# Stop the server before rebuilding or manual cleanup
./scripts/cleanup.ps1
```

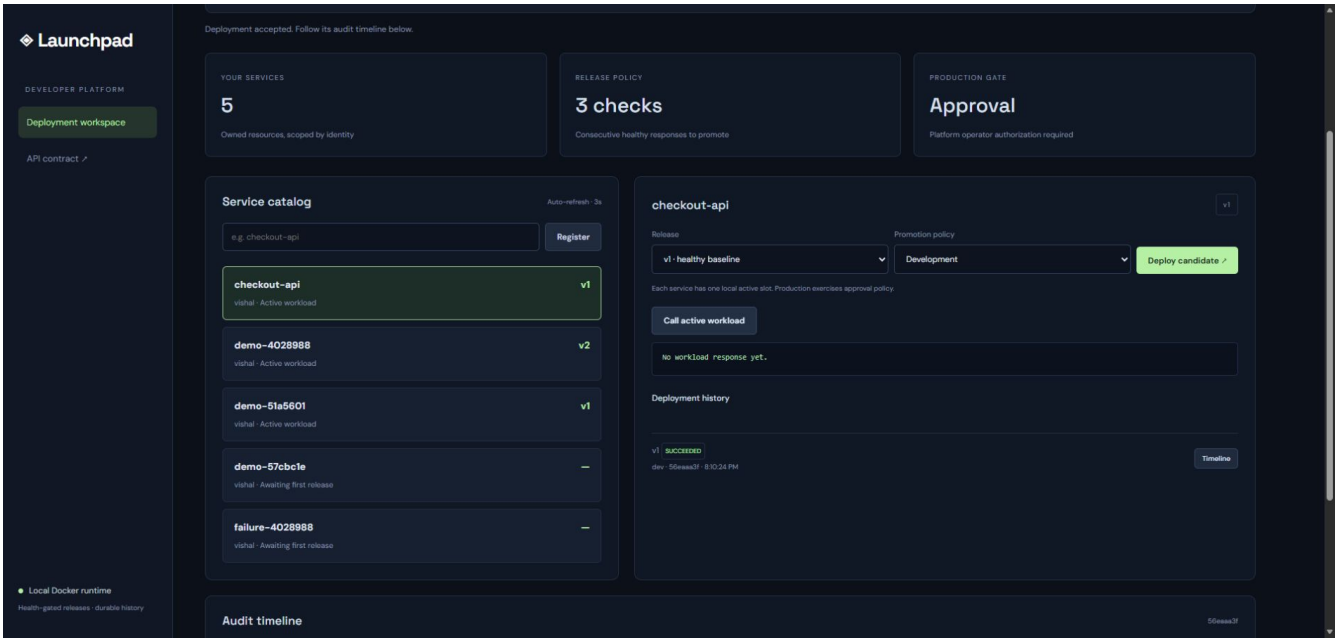
Default API port is 8088; isolated acceptance uses 8091 and recovery uses 8090. Docker assigns ephemeral workload host ports. The dashboard accepts the documented local demo token. Configure distinct random environment tokens for any non-demo use; these static sample credentials are public fixtures, not real secrets.

| Symptom | Check and explanation |
|---|---|
| Java loopback exception | Use start.ps1. Its non-existent NIO fallback path addresses this Windows launch environment. |
| Maven cannot rename jar | Stop the running native process before repackaging on Windows. |
| Candidate fails unexpectedly | Inspect Docker logs, gate debug logs, CPU and cold-start timing. Retained route is the intended failure behavior. |
| Five-service quota | Reuse a service or run the isolated acceptance script with a fresh database/namespace. |
| Active response is 502 | Check whether the recorded container is running; the platform does not automatically heal an active target. |
| Ports work natively but not in container | Check loopback namespaces and documented host-network requirements. |

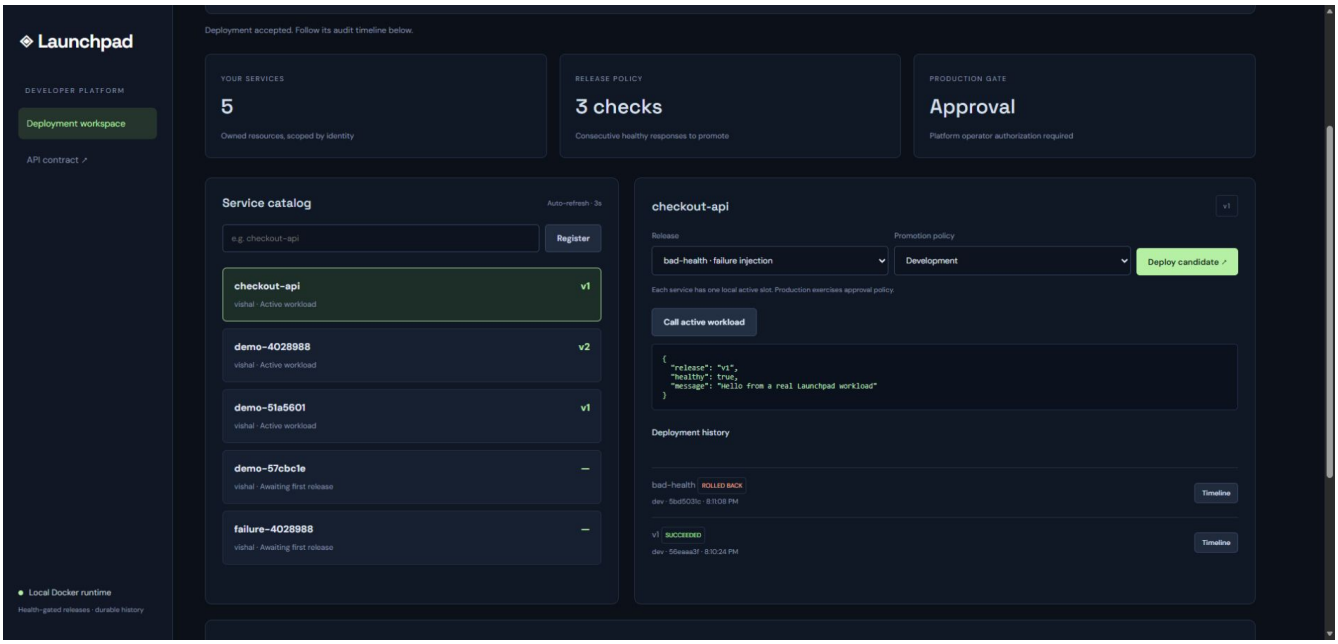
Manual cleanup preserves SQL history while removing labelled containers. That can leave active fields pointing at removed objects. A new deployment or an intentionally fresh database/namespace restores a coherent demonstration; do not call deletion of runtime state a rollback.

# See a healthy release survive failure

These are genuine screenshots from the local checkout-api demonstration on 14 October 2026. The API, history and response panels describe actual Docker workload behavior.



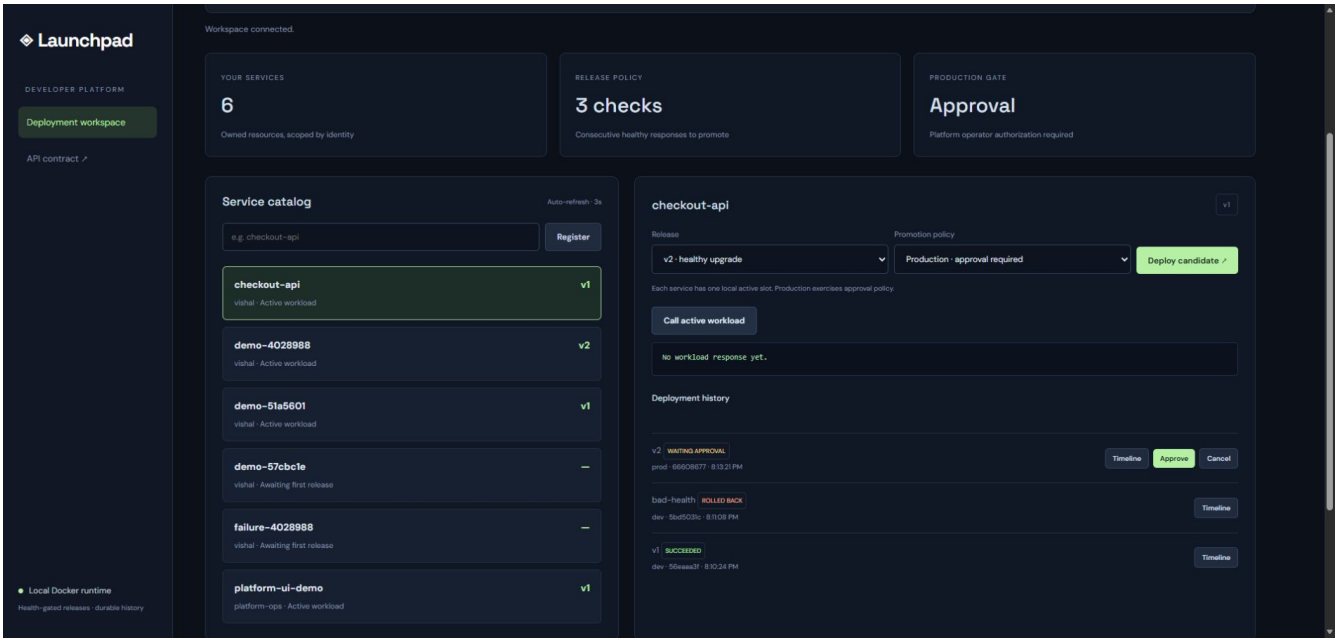

Healthy baseline v1 is selected and its deployment is SUCCEEDED. The response is visible in the next scenario.



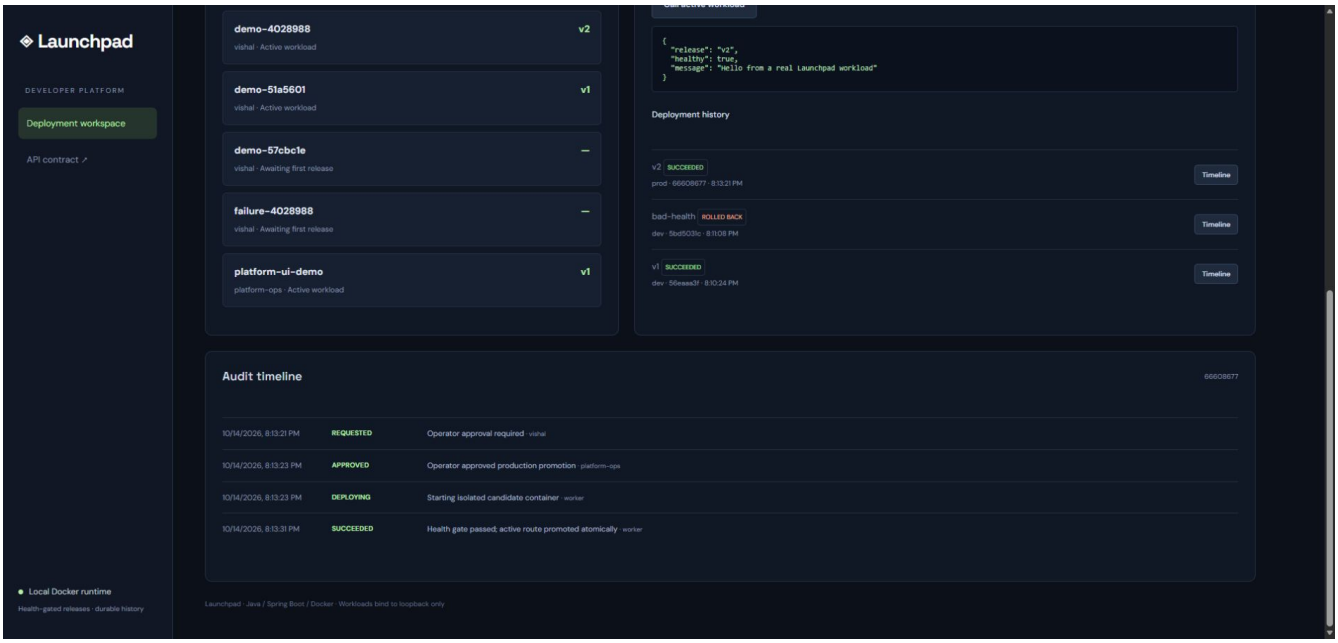

Failure injection bad-health became ROLLED BACK while the active response remained healthy v1.

When presenting, point first to the selected service, then the release state, then the response. A screenshot illustrates the result; the acceptance script and audit records provide stronger reproducible evidence.

# See approval and attributable promotion




Approval gate the operator can approve a WAITING APPROVAL intent while v1 remains the selected target.




Attribution APPROVED records platform-ops, followed by DEPLOYING and SUCCEEDED. The workload response is now v2.

The developer approval rejection is established by API tests, not by hiding a button. The screenshots also show the precise policy scope: one local active slot per service. Explain that scope before discussing a protected production environment design.

# Interview questions on product and design

## 1 What did you build

I built a Java and Spring Boot control plane for developer-requested local deployments. It persists intent, applies ownership and approval rules, starts real Docker candidates and promotes only after a health gate. I verified retries, failed upgrades, recovery and infrastructure convergence rather than inventing impact metrics.

## 2 Why is this more than a CRUD application

The SQL records coordinate external side effects and enforce lifecycle invariants. The interesting behavior is what happens after an ambiguous retry, conflicting request, failed health gate or process crash. The tables support those decisions; they are not the project's end goal.

## 3 Why return 202

A committed request can wait for approval, queueing or runtime work. Returning 202 exposes that asynchronous contract. The deployment ID gives the client a durable status resource instead of coupling it to a long-lived HTTP request that may disconnect during startup.

## 4 Why one worker and H2

They make a reproducible local system feasible without a broker or managed database. I kept transactions explicit and tested important failure paths. I would not scale replicas by simply replacing the JDBC URL; distributed ownership, leases and external-effect fencing are separate design work.

## 5 Is this Kubernetes or a replacement for it

No. The current runtime uses Docker on one local host and a deliberately fixed sample image. The project demonstrates lifecycle coordination and a developer path. A Kubernetes adapter would still need deployment ownership, readiness, rollout status and reconciliation semantics.

## 6 What is the control plane and data plane distinction

The control plane accepts and coordinates desired changes. The workload containers serve the application. Launchpad's demo response endpoint proxies through the control plane, so those paths are not fully separated here. A production ingress should allow existing workloads to serve independently of orchestration availability.

## 7 Does production mean a deployed production environment

No. prod selects operator approval for an intent on the same local service slot used by dev. There is no separate environment resource or cloud boundary. That makes the approval behavior demonstrable, but stronger production governance requires protected environment identities and policy.

## 8 What is your strongest evidence

The recorded evidence includes 17 integration tests, nine real acceptance checks, same-container recovery after an abrupt process kill, and a local Terraform apply/no-drift/destroy cycle. GitHub CI separately passed tests, both image builds and live Docker acceptance for the linked source commit.

# Interview questions on APIs and SQL

### 9 What if the client retries after a timeout

The key may already identify a committed deployment. The API looks up authenticated requester plus key and compares service, release and environment. A matching retry returns that deployment's UUID and current state; a mismatching payload returns 409. The retry must preserve its key.

### 10 What if the same key is used across services

Its scope is per requester across all services, not per service. A database unique constraint covers that pair. Two transactions can lock different service rows, so the uniqueness check and duplicate-key recovery remain important even though same-service requests are serialized.

### 11 Why lock the service row

The busy-state check and insert must behave as one decision for that service. Otherwise two requests can both see no active intent and insert competing deployments. SELECT FOR UPDATE serializes the check with other submissions, approval, cancellation and worker claim transactions.

### 12 Why not use synchronized everywhere

A Java monitor coordinates threads in one object instance. It does not protect another JVM or make a database decision atomic. Launchpad uses it for its single worker and SQL row locks for resource decisions. A worker fleet would need durable ownership and fencing.

### 13 Can the service quota race

The implementation locks one quota row before count-and-insert, preventing two registrations from both using the last slot. It serializes all registrations, including different principals. A per-owner counter or per-owner lock would reduce that contention while preserving the quota invariant.

### 14 Why not hold the transaction during health checks

That would hold locks while external operations wait and fail. Instead, the worker records a claim, performs runtime work outside SQL transactions, and commits the result later. The intervening crash windows are handled through deterministic identity and reconciliation, not distributed SQL rollback.

### 15 Is the API fully idempotent

Intent creation is idempotent for matching key and payload. Repeated cancellation of CANCELLED work also returns that record. Approval only replays cleanly while already QUEUED; after later states it can conflict. These are endpoint-specific behaviors, not a universal exactly-once API guarantee.

### 16 Why return 404 for another owner

It avoids confirming that the resource exists while denying access. The same check covers service reads, writes and event timelines. Operators intentionally have wider access. This is principal ownership isolation, not a complete organization and tenant membership model.

# Interview questions on failures and recovery

### 17 What happens if the worker dies after Docker run

The DEPLOYING intent already stores a candidate name derived from its UUID. After restart the worker inspects that name, validates its namespace and reuses the candidate, then repeats health checks. The abrupt-kill scenario verified the same real Docker container ID after recovery.

### 18 Is this exactly once execution

No. A worker can repeat inspection, start and health checks. The design makes creation identity stable and recovery converge toward one durable result. A process crash can also leave metrics out of sync with history. Exactly-once external execution is not the project's claim.

### 19 Why three consecutive probes

One success can be transient. A consecutive streak resets after any failed probe and prevents promotion on an isolated healthy response. Three is a small demo policy, not a statistically established reliability threshold. A real service needs a readiness contract and a justified observation window.

### 20 What if the previous release is also unhealthy

The worker preserves the stored prior target and calls the outcome ROLLED_BACK if that field exists. It does not recheck that target during failure handling. I would describe this as retaining the last promoted route and add explicit active-target monitoring before claiming healthy recovery.

### 21 Why do you not claim zero downtime

A response request can read the old port immediately before cleanup removes that container. There is no drain interval or connection-aware ingress. Health-gated target changes reduce one rollout risk, but zero downtime would need serving-path design and appropriate traffic tests.

### 22 What if Docker times out but later creates a container

Killing the CLI does not undo work already accepted by the daemon. Persisted candidate identity and cleanup reconciliation help locate it. A production runtime should distinguish an unknown operation result from a confirmed failure, persist remote operation identifiers and test delayed completion explicitly.

### 23 What happens if cleanup fails

The deployment outcome remains durable, but no CLEANED event is appended. A later scan can try removal again. This decouples success or rollback from retirement. Backoff, an indexed cleanup queue and bounded history scans would be needed to make that mechanism efficient at scale.

### 24 What if an active workload dies later

The current system does not heal it automatically. The active response endpoint returns 502 when the target cannot provide a successful response. Candidate health gating and continuous active-workload reconciliation are different responsibilities. Monitoring, replacement policy and escalation would be a separate enhancement.

# Interview questions on security and runtime

### 25 How are users authenticated

An explicit bearer token is compared against three configured demo tokens and resolves to a principal and role for the request. The tokens are distinct, at least 24 characters and have no expiry workflow. I would use OIDC-backed short-lived credentials and proper rotation in a deployed platform.

### 26 Why is CSRF disabled

The app expects credentials supplied explicitly in an Authorization header, without cookies, sessions or Basic authentication. The relevant property is absence of automatically attached browser credentials, not merely being stateless. If the authentication model changes, the CSRF decision must be revisited.

### 27 Does hiding Approve secure the operation

No. The server requires ROLE_OPERATOR for the approval route. The dashboard reflects the role for usability, while integration and acceptance checks verify a developer receives 403. Client-side visibility is not an authorization boundary.

### 28 How is command injection prevented

The client cannot supply arbitrary image names, shell commands, mounts or environment variables. Releases are validated, SQL uses parameters and Docker is invoked with a fixed ProcessBuilder argument list rather than a shell. Administrator-controlled image configuration still belongs to the trusted runtime boundary.

### 29 Is the image immutable

The API fixture uses a local tag and APP_VERSION, so no. --pull=never prevents a deployment-time pull but does not freeze the tag. The independent Terraform module accepts a sha256 image ID. A production artifact catalog should resolve approved releases to verified immutable digests.

### 30 Does non root make Docker access safe

It constrains the workload process, but daemon access can still provide broad authority over the host runtime. Resource limits, dropped capabilities and read-only root reduce specific risks; they do not make this a safe general platform for mutually untrusted tenants.

### 31 Are labels an access control system

They are an ownership guard against accidental reuse or deletion across namespaces. A host administrator can spoof them. The current reuse check matches the namespace; it is not cryptographic artifact attestation. Stronger isolation must come from the runtime and identity boundary.

### 32 Can you enforce two person production approval

The current operator role controls approval, but operators can request and approve their own intents. To require two people, persist requester and approver identities, reject equality and protect the destination environment independently of the request's dev or prod field.

# Interview questions on scale and cloud

### 33 How would you add more workers

Use a shared transactional database, bounded claims, lease owner and expiry, attempt records and fencing tokens. Lock or claim by service to preserve promotion order. The external runtime must reject stale operations or serialize them. Adding threads alone does not create safe distributed coordination.

### 34 Would you add Kafka

Only if notification, fan-out or throughput requirements justify it. A database queue is sufficient for the current local scope. With a broker, use an outbox to avoid committing intent without publishing notification; consumers still need deduplication, durable ownership and runtime reconciliation.

### 35 What would you index or paginate first

Use a targeted active-intent existence query, a stable queue order and an indexed pending-cleanup table. Paginate service deployment history and audit events, then define retention. Current scans and N-plus-one event lookups are more immediate scaling issues than the choice of UI framework.

### 36 How would Azure fit

Implement a real runtime adapter using managed identity and ARM operations, persist operation IDs, poll status, probe a candidate slot and promote routing. Reconcile partial completion and manage private image pulls. The current Azure Terraform file is only a validated workload reference, not that adapter.

### 37 What would your first SLO measure

Define user-facing acceptance and deployment completion separately. Instrument queue wait, approval wait, runtime stage duration and failure reasons. Decide which waits the platform owns before calculating an SLI. The current timer does not measure full request-to-ready latency, and no target is measured or claimed.

### 38 What is the backup and disaster recovery plan

The current H2 file is local state without an implemented backup or tested host-loss recovery policy. A production design would define RPO and RTO, back up the shared database, test restores, and reconcile retained runtime objects against restored intent and route records.

### 39 What would you improve first

Separate telemetry failure handling from committed promotion, test ambiguous runtime outcomes and graceful shutdown, then improve queue and cleanup queries. Next add protected environments, immutable artifacts and identity integration. Multi-worker throughput should follow correctness tests and observability, not precede them.

### 40 Why did you leave AI out

The project's core value is reliable lifecycle coordination. An LLM is not needed to decide whether an HTTP health gate passed. A later assistant could summarize audit evidence or suggest diagnostics, while deterministic policy and authorized actions remain outside the model's discretion.

# Explain the project aloud

## A thirty second introduction

I built Launchpad, a Spring Boot control plane for self-service local deployments. It stores deployment intent, enforces ownership and production approval, and runs real Docker candidates. A release becomes active only after three consecutive health checks. A failed candidate preserves the previous route. I verified concurrent retries, failure paths and recovery after killing the worker, and published reproducible evidence with passing GitHub CI.

## A deeper explanation in about two minutes

The main design decision is to separate accepting a request from executing it. The API returns 202 only after a transaction stores the intent and audit event. It locks the service row so competing requests cannot both create active work. A requester-scoped idempotency key makes an ambiguous client retry return the same intent.

The scheduled worker claims that intent, performs Docker calls outside SQL transactions and probes the candidate. It persists the candidate name before execution, so a restart can inspect and reuse the same container. Promotion updates the active target and terminal status together. Cleanup is reconciled afterwards, allowing removal failures to retry without changing the deployment result.

The implementation intentionally uses one worker and file-backed H2 for reproducibility. Static bearer principals and a fixed sample image make the security and approval behavior testable. For a deployed platform I would add OIDC, immutable artifacts, a separate ingress, protected environments and a shared database with leases and fencing. The Azure reference is validated infrastructure code; I have not deployed an Azure adapter.

## A five minute live demo

| Time | Show | Explain |
|---|---|---|
| 0 to 1 min | Register service and deploy v1 | Persisted intent, status and real response |
| 1 to 2 min | Deploy bad-health | Old target stays selected during the gate |
| 2 to 3 min | Inspect ROLLED_BACK and CLEANED | Failure result and cleanup are separate |
| 3 to 4 min | Request prod v2 and approve as operator | Server authorization and attributable approval |
| 4 to 5 min | Show recovery evidence and architecture | Crash window, same container and next tradeoff |

When challenged, use a concrete request or crash window instead of listing technologies. If a proposed feature is not implemented, say how you would add it and what you would test before claiming it.

# Glossary and source references

| Term | Plain meaning in this project |
|---|---|
| Intent | A persisted request describing the change to attempt. |
| Invariant | A property that must remain true through concurrency or failure. |
| Reconciliation | Compare durable state with runtime objects and repeat safe work. |
| Idempotency | A matching retry refers to the same durable request. |
| Atomic promotion | Update selected target and terminal state in one SQL commit. |
| Lease and fencing | Proposed time-bound ownership plus rejection of stale workers. |
| Outbox | Proposed durable notification record committed with the intent. |
| Readiness and liveness | Able to serve the required workload / process still functioning. |
| RPO and RTO | Acceptable data-loss window / target restoration time. |

## Primary project sources

Repository and runnable setup | Source snapshot 7b11f3eb1a35dd23a7f833336fd84967cdffd949

API and validation

Transactions and policy

Worker and runtime

Runtime side effects

Schema and indexes

Passing GitHub CI run 38583306582 | Tested application source cd73304. See docs/verification.md and evidence/summary.json for local checks.

## Official background references

Spring Security on CSRF explains the role of automatically supplied browser credentials. Docker Engine security explains the daemon trust boundary. PostgreSQL SELECT locking clauses provide background for the proposed shared work-claim design. These are reference concepts, not evidence that the proposed production changes are implemented.

Technical descriptions were checked against the local source on 20 October 2026. Screenshots and execution evidence are from 14 October 2026. No new performance, production availability or Azure deployment claim is made.